

PHASE 1 SUMMARY

FastAPI旅客安检接口项目 第一阶段总结

从基础接口练习到可维护、可测试的小型后端项目

第一阶段完成了一套基于 FastAPI、SQLAlchemy、SQLite 和 Alembic 的旅客安检信息管理 API，具备 CRUD、数据校验、分页筛选、统一异常、API Key 鉴权、请求日志、数据库迁移和自动化测试能力。

项目目录：F:\shixi\api_practice

阶段状态：已完成，自动化测试 14 passed

一、阶段目标

从零搭建一套可运行、可维护、可测试的旅客安检信息接口系统，并将最初的单文件练习代码逐步升级为结构较完整的小型后端项目。

二、核心功能实现

旅客信息 CRUD
支持新增、精确查询、更新、删除和列表查询。

数据持久化
由内存列表升级为 SQLite，并最终使用 SQLAlchemy ORM。

分页筛选排序
支持分页、姓名模糊查询、多条件筛选和升降序排列。

参数校验
校验航班号、日期、座位号、必填字段和备注长度。

安全鉴权
旅客接口统一要求 X-API-Key。

工程化能力
包含 Alembic 迁移、日志、环境配置和自动化测试。

1. 接口清单

方法	路径	作用
POST	/security/passenger	新增旅客安检信息
GET	/security/passenger	按证件号、航班号和航班日期精确查询
PUT	/security/passenger	更新座位号、安检结果、行李状态、备注等
DELETE	/security/passenger	删除指定旅客航班记录
GET	/security/passengers	分页查询旅客列表

系统使用 **certNo + fltNo + fltDate** 作为业务唯一条件，从而允许同一旅客拥有不同航班或不同日期的记录，同时拦截重复提交。

2. 列表查询能力

- 分页参数：page、pageSize
- 筛选条件：证件号、航班号、航班日期
- 姓名模糊查询：psrName
- 排序字段：id、createdAt

- 排序方向：asc、desc
- 返回字段：list、page、pageSize、total、totalPages

3. 数据校验

- 航班号仅允许字母和数字，并自动转为大写。
- 航班日期必须为 YYYY-MM-DD，且必须是真实日期。
- 座位号格式类似 12A、16B、108C。
- 必填字段不能是空字符串。
- 备注最多 200 个字符，可新增、修改和清空。

三、数据库与迁移

数据存储经历了以下演进：

```
Python 内存列表
→ SQLite + sqlite3
→ SQLite + SQLAlchemy 2.x ORM
```

数据库表中包含 id、cert_no、flt_no、flt_date、data_json、created_at、remark，并通过联合唯一约束保证同一旅客、同一航班、同一日期不重复。

Alembic 迁移

- 创建 passengers 初始表迁移。
- 将已有数据库通过 stamp 纳入 Alembic 管理。
- 完成 remark 字段的真实迁移升级。
- 当前迁移版本：cc0c73bee5e9 (head)。

```
修改 models.py
→ alembic revision --autogenerate
→ 人工检查迁移脚本
→ alembic upgrade head
→ pytest -v
```

四、安全、配置与日志

1. API Key 鉴权

- 请求头名称：X-API-Key
- 不传密钥：401
- 密钥错误：403
- 密钥正确：正常访问
- Swagger /docs 支持 Authorize 授权测试

2. .env 配置管理

使用 pydantic-settings 管理 APP_NAME、DATABASE_URL 和 API_KEY，避免将可变配置和敏感信息直接写进业务代码。

```
APP_NAME=接口练习项目
DATABASE_URL=sqlite:///./passenger.db
API_KEY=真实密钥
```

3. 请求日志

每次请求会记录方法、路径、状态码和服务端处理耗时，同时在响应头中返回 X-Process-Time-Ms。

```
GET /security/passengers status=200 duration=21.68ms
GET /security/passenger status=404 duration=3.86ms
```

五、统一响应与异常处理

```
{
  "code": "业务码",
  "message": "处理结果",
  "data": null
}
```

HTTP 状态码	含义
400	业务参数错误
401	缺少 API Key
403	API Key 错误
404	数据不存在
409	重复数据
422	请求参数校验失败

六、自动化测试

项目使用 pytest、FastAPI TestClient 和临时 SQLite 测试数据库，共完成 14 个自动化测试，覆盖：

- 新增、重复新增、精确查询、更新和删除
- 日期和请求参数校验
- 分页、姓名筛选和排序
- 备注新增、查询、更新和清空

- 缺少 API Key 和错误 API Key

当前回归测试结果：14 passed

七、项目结构

```
api_practice/
├─ app/
│   ├─ routers/
│   │   ├─ airport.py
│   │   └─ passenger.py
│   ├─ config.py
│   ├─ database.py
│   ├─ exception_handlers.py
│   ├─ main.py
│   ├─ models.py
│   ├─ request_logging.py
│   ├─ schemas.py
│   └─ security.py
├─ migrations/
├─ main.py
├─ test_main.py
├─ alembic.ini
├─ requirements.txt
├─ .env
├─ .env.example
└─ README.md
```

八、完整业务链路

外部系统发送旅客安检信息

- API Key 鉴权
- 请求体解析
- Pydantic 数据校验
- SQLAlchemy 写入数据库
- 联合唯一约束防止重复
- 返回统一响应
- 记录状态码和处理耗时

九、常用命令

启动项目

```
cd F:\shixi\api_practice
.\.venv\Scripts\Activate.ps1
alembic upgrade head
fastapi dev main.py
```

运行测试

```
pytest -v
```

查看数据库迁移版本

```
alembic current
```

接口文档

```
http://127.0.0.1:8000/docs
```

十、阶段结论

第一阶段已经完成一套基于 FastAPI、SQLAlchemy、SQLite 和 Alembic 的旅客安检信息管理 API。系统具备数据校验、持久化、CRUD、分页筛选、统一异常、API Key 鉴权、日志、数据库迁移和自动化测试能力，已经从基础练习代码发展为结构完整、可继续扩展的小型后端项目。

第一阶段状态：完成。后续可按实际需求选择 MySQL、服务器部署、前端界面或 JWT 权限体系等方向继续扩展。